SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

(a) Analyze the possible reasons for lack of responsiveness in the existing layout.
(b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
(c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
(d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
(e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

(a) Analyze the issues in the existing form validation mechanism.
(b) Design input fields for email and phone number with appropriate HTML attributes.(c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
(d) Develop a mechanism to display **real-time error messages** for invalid inputs.
(e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

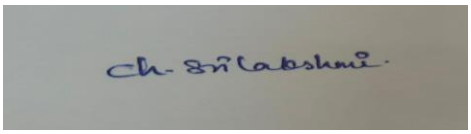
Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

(a) Analyze the CSS properties responsible for layout overlapping issues.
(b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
(c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
(d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
(e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

I Ch. Sri Lakshmi/192311342(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

ANSWERS

Experiment 1: Responsive Layout Design

(a) Analyze the possible reasons for lack of responsiveness.

Answer:

The webpage is not responsive because:

1. Fixed width values (e.g., width:1000px) are used.
2. No CSS Media Queries are implemented.
3. Images have fixed sizes instead of responsive sizes.
4. CSS Flexbox or Grid is not used.
5. Margins and padding are not adjusted for different screens.
6. The viewport meta tag is missing.

(b) Design a responsive webpage layout using CSS Flexbox.

HTML

HTML

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<link rel="stylesheet" href="style.css">
</head>

<body>

<header>Retail Website</header>

<nav>
<a href="#">Home</a>
<a href="#">Products</a>
<a href="#">Offers</a>
<a href="#">Contact</a>
</nav>

<div class="container">

<div class="content">
<h2>Products</h2>
<p>Responsive webpage using Flexbox.</p>
</div>
```

```
<div class="sidebar">
<h3>Offers</h3>
<p>Today's Discounts</p>
</div>

</div>

<footer>
Copyright © 2026
</footer>

</body>
</html>
```

CSS

CSS

```
body{
margin:0;
font-family:Arial;
}

header,footer{
background:#333;
color:white;
text-align:center;
padding:20px;
}

nav{
display:flex;
justify-content:center;
background:#555;
}

nav a{
color:white;
padding:15px;
text-decoration:none;
}

.container{
display:flex;
padding:20px;
gap:20px;
}

.content{
flex:3;
background:#f2f2f2;
```

```
padding:20px;
}

.sidebar{
flex:1;
background:#ddd;
padding:20px;
}
```

(c) Demonstrate responsiveness using Media Queries.

CSS

```
@media(max-width:768px){

.container{
flex-direction:column;
}

nav{
flex-direction:column;
}

}
```

Output

Desktop

- Header
- Navigation (Horizontal)
- Content and Sidebar side-by-side
- Footer

Tablet

- Reduced spacing
- Sidebar below content

Mobile

- Navigation becomes vertical
- Content occupies full width
- Sidebar appears below content

(d) Implement spacing, alignment and distribution.

CSS

```
.container{
display:flex;
gap:20px;
justify-content:space-between;
align-items:flex-start;
padding:20px;
}

.content,
.sidebar{
```

border-radius:8px;
}
Improvements

- Proper spacing
- Better alignment
- Equal distribution
- Attractive layout

(e) Evaluate the design.
Advantages

- Works on all devices
- Better readability
- Proper spacing
- Easy navigation
- Improved user experience

Further Improvements

- Use CSS Grid for complex layouts.
- Optimize images.
- Add animations.
- Improve accessibility.
- Use responsive typography.

Experiment 2: Form Validation using JavaScript
(a) Analyze the issues.
Problems:

- Invalid email accepted.
- Invalid phone number accepted.
- No real-time validation.
- Incorrect data stored.
- Poor user experience.

(b) Design HTML input fields.
HTML
<form>

Email:

<input type="email" id="email" required>

Phone:

<input type="text"
id="phone"
maxlength="10"
required>

<button onclick="validateForm()">
Submit

```
</button>

</form>

<p id="message"></p>
```

(c) Implement JavaScript Validation.

JavaScript

```
function validateForm(){

let email=document.getElementById("email").value;
let phone=document.getElementById("phone").value;

let emailPattern=/^[^\s@]+@[^\s@]+\.[^\s@]+$/;

let phonePattern=/^[0-9]{10}$/;

if(!emailPattern.test(email)){
alert("Invalid Email");
return false;
}

if(!phonePattern.test(phone)){
alert("Invalid Phone Number");
return false;
}

alert("Registration Successful");
```

(d) Display real-time error messages.

JavaScript

```
document.getElementById("email").addEventListener("input",function(){

let pattern=/^[^\s@]+@[^\s@]+\.[^\s@]+$/;

if(!pattern.test(this.value))
document.getElementById("message").innerHTML="Invalid Email";
else
document.getElementById("message").innerHTML="";

});
```

Similarly for Phone Number:

JavaScript

```
document.getElementById("phone").addEventListener("input",function(){

let pattern=/^[0-9]{10}$/;

if(!pattern.test(this.value))
```

```
document.getElementById("message").innerHTML="Invalid Phone";
else
document.getElementById("message").innerHTML="";

});
```

(e) Evaluate the validation system.

Advantages

- Prevents invalid inputs
- Reduces server load
- Better user experience
- Faster error correction

Enhancements

- Server-side validation
- Password strength checker
- CAPTCHA
- OTP verification
- Input sanitization

Experiment 3: CSS Layout Debugging

(a) Analyze CSS properties causing overlap.

Reasons:

- Fixed widths and heights
- Incorrect absolute positioning
- Float without clear
- Missing Flexbox/Grid
- Overflow hidden
- Incorrect z-index

(b) Explain the CSS Box Model.

The CSS Box Model consists of:

1. **Content** – Actual text or image.
2. **Padding** – Space inside the border.
3. **Border** – Surrounds the padding.
4. **Margin** – Space outside the border.

Diagram:

(c) Debug using positioning.

CSS
```
.container{
display:flex;
flex-wrap:wrap;
}

.box{
position:relative;
width:300px;
margin:10px;
}
```
Or using Grid:
CSS
```
.container{
display:grid;
grid-template-columns:repeat(auto-fit,minmax(250px,1fr));
gap:20px;
}
```

(d) Modify CSS.
CSS
```
*{
box-sizing:border-box;
}

img{
max-width:100%;
height:auto;
}

.container{
display:flex;
flex-wrap:wrap;
gap:20px;
}

@media(max-width:768px){

.container{
flex-direction:column;
}

}
```

(e) Evaluate the corrected layout.
Results

- No overlapping
- Responsive design

- Better alignment
- Consistent spacing
- Improved usability

Best Practices

1. Use Flexbox or Grid.
2. Avoid fixed widths.
3. Use media queries.
4. Use box-sizing: border-box.
5. Test on multiple devices.
6. Optimize images.
7. Write clean and modular CSS.